

TERM	COURSE NAME	COURSE CODE	VERSION
2261	Introduction to Programming Using C	IPC144	A

Name	
Student Number	(write your student number here)
Section	NAA

Date: February 20, 2026
Duration: 2 hours
Total Marks: 50 marks
Weight: 35%
Professor's Name Catherine Leung

SPECIAL INSTRUCTIONS:

1. Place all personal electronic equipment (phones/watches/earphones etc.) into your bags. Bags must be either placed **under** your chair or at front of room. Having a personal device on your person, not inside your bags will be considered an academic integrity violation.
2. Large jackets must also be placed at the front of the room, not on your chair
3. Write your answers in the spaces provided
4. Communication is only allowed between you and your professor and other invigilators. Communication with your friends and classmates are strictly forbidden

This exam includes this *cover page*, plus 8 pages of questions.

SENECA'S ACADEMIC INTEGRITY POLICY

As a Seneca student, you must conduct yourself in an honest and trustworthy manner in all aspects of your academic career. A dishonest attempt to obtain an academic advantage is considered an offense and will not be tolerated by the College.

By beginning this test, I affirm that I will not give or receive any unauthorised help in this test and that all work provided will be my own. I agree to abide by Seneca's Academic Integrity Policy, and I understand that any violation of academic integrity will be subject to the penalties outlined in the policy.

Question 1: Debugging (10 marks)

The following function is supposed to prompt the user to repeatedly enter positive numbers. An entry of a 0 indicates that the input is complete. Negative numbers are ignored with an error message "Only positive numbers are allowed". The function will return the smallest number entered by the user (excluding the 0 that was used to stop the loop). If no values are entered (user just hits 0 right away), the function return -1 to indicate an error.

```

1. int getSmallest(void)
2. {
3.     int smallest;
4.     int input;
5.     printf("Enter a positive number (0 to stop): ");
6.     scanf("%d", input);
7.     if(input != 0){
8.         smallest = input;
9.     }
10.    while (input > 0){
11.        if (input > 0)
12.            if(input < smallest){
13.                input = smallest;
14.            }
15.
16.        else{
17.            printf("Only positive numbers are allowed\n");
18.        }
19.        printf("Enter a positive number (0 to stop): ");
20.        scanf("%d", input);
21.    }
22.
23. }
```

However, there are bugs in this program. Fill in the following table to indicate line with bugs, whether it is (S)yntactic or (L)ogical, a short description and the corrected code. Table might have more than required number of rows

Line #	(S)yntactic or (L)ogical,	Brief Description	Fix

Question 3: Programming (20 marks)

The programming question for this test is divided into a few functions and a final programming problem. Each function will be assessed independently. The overall program solution will need to use these functions where appropriate. Even if the code in your function is not fully correct, demonstrating the ability to call and use the functions in an appropriate manner will provide you with full marks for the programming problem.

In addition to the functions you are asked to write, you will also going to be provided with two functions that have already been coded for you. **You can use these functions without implementation.** Details will be provided later.

You have been hired to work on a pizza ordering system. The system will determine the price of an order and an estimated time when the pizza will be ready.

Q3a: Write the following function (3 marks)

```
double pizzaPrice(int regular, int premium);
```

This function is passed two numbers representing the number of *regular* and *premium* toppings and returns the price of the pizza. This pizza company only sells pizzas of one size. Thus, the cost is determined only by the number of each type of toppings and the base price.

- The base price of the pizza is \$10.00
- Regular toppings cost \$1.00 each
- Premium toppings cost \$1.50 each

```
double pizzaPrice(int regular, int premium){
```

```
}
```

Q3b: Write the following function (3 marks)

```
double deliveryPrice(int distance);
```

This function is passed the *distance* in kilometers. It will return the delivery price for the given distance

- * 3 km or less - \$5.00
- * more than 3 km but less than 5 km - \$ 6.00
- * 5 or more km - \$8.00

```
double deliveryPrice(int distance){
```

```
}
```

Q3c: Write the following function (7 marks)

```
void printReceipt(int currHour, int currMin, int numPizza, double delivery, double pizzaPrice);
```

This function is passed:

- the current time using the twenty four hour clock (**currHour** and **currMin**. example currHour = 13 and currMin = 55 would mean 13:55)
- the number pizzas in the order (**numPizza**)
- delivery charge (**delivery** - if this value is 0, it means there is no delivery)
- The pizza price of the order (**pizzaPrice**)

This function prints a receipt based on the provided information.

You will need to work out when the pizza(s) are ready for pickup to do this. The restaurant has an oven that can hold 4 pizzas at a time. Each pizza takes 6 minutes to bake. Thus, they can make at most 4 pizzas every 6 minutes. 3 pizzas will take 6 minutes, 5 would take 12, 16 pizzas would take 24 min. Add this prep time to the current time to find the time when the pizzas are ready. You may assume that the ready time will never exceed 23:59 because the store closes by 22:00

The receipts prints "Pizza" with a number one per line for each pizza, the total pizza price, the delivery price and the total price and the time when the pizza will be ready. See examples below for exact format.

Note that the Delivery Price line and "**+ delivery time**" is only printed if the order is for delivery. "**+ delivery time**" is just a fixed string not an actual time.

Examples:

```
//curr time is 13:55, 5 pizzas (takes 12 min to make 5 pizzas),
//5.00 delivery charge, 52.50 pizza price
printReceipt(13, 55, 5, 5.00, 52.50); // this function call would print:
```

```
IPC PIZZA STORE
```

```
Pizza 1
Pizza 2
Pizza 3
Pizza 4
Pizza 5
```

```
Pizza Price:    $52.50
Delivery Price: $5.00
Total Price:    $57.50
Ready by:       14:07 + delivery time
```

```
//curr time is 14:47, 9 pizzas (takes 18 min to make 9 pizzas),
//0 delivery charge (indicating it is for pickup not delivery), 90.00 pizza price
printReceipt(14, 47, 9, 0.00, 90.00); // this function call would print:
```

```
IPC PIZZA STORE
```

```
Pizza 1
Pizza 2
Pizza 3
Pizza 4
Pizza 5
Pizza 6
Pizza 7
Pizza 8
Pizza 9
```

```
Pizza Price:    $90.00
Total Price:    $90.00
Ready by:       15:05
```

Please implement this question on the next page

Seneca
POLYTECHNIC | SCHOOL OF COMPUTER
PROGRAMMING & ANALYSIS

Q3d: Write the following program. (7 marks)

Write the following program making use of the 3 functions you wrote previously as well as the following two functions. Note that you do NOT have to implement these functions, only call them as needed.:

int getCurrHour(void); – returns the current time's hour component in 24 hour time
int getCurrMinute(void); – returns the current time's minute component in 24 hour time.

This program will ask the user for the delivery distance using 0 to indicate that the pizza is to be picked up. It will then repeatedly ask the user to enter the number of regular toppings and premium toppings they want on a pizza, and whether they want to order another pizza using Y/N as possible inputs. You may assume that the inputs will be valid. The function will then print out the receipt.

Use the sample run below for exact prompt text and formatting. (note that the current time was obtained using the getCurrHour() and getCurrMinute() functions. In example output, the current time was 13:55). **Text in bold** is produced by the program. Faded text is entered by the user:

WELCOME to IPC Pizza Store

```
What is your delivery distance in km (0 for pickup): 2
How many regular toppings on pizza 1? 1
How many premium toppings on pizza 1? 1
Would you like to order another pizza (Y or N)? Y
How many regular toppings on pizza 2? 0
How many premium toppings on pizza 2? 0
Would you like to order another pizza (Y or N)? Y
How many regular toppings on pizza 3? 0
How many premium toppings on pizza 3? 0
Would you like to order another pizza (Y or N)? Y
How many regular toppings on pizza 4? 0
How many premium toppings on pizza 4? 0
Would you like to order another pizza (Y or N)? Y
How many regular toppings on pizza 5? 0
How many premium toppings on pizza 5? 0
Would you like to order another pizza (Y or N)? N
```

IPC PIZZA STORE

```
Pizza 1
Pizza 2
Pizza 3
Pizza 4
Pizza 5
```

```
Pizza Price:    $52.50
Delivery Price: $5.00
Total Price:    $57.50
Ready by:       14:07 + delivery time
```

Please implement this question on the next page

```
#include <stdio.h>
int getCurrHour(void);    // returns the current time's hour component in 24 hour time
int getCurrMinute(void); //returns the current time's minute component in 24 hour time
double pizzaPrice(int regular, int premium);
double deliveryPrice(int distance);
void printReceipt(int currHour, int currMin, int numPizza, double delivery, double pizzaPrice);

int main(void){
```

```
}
```

Question 4: Walkthrough [20 Marks]

- a) What is the exact output of the following program? (12 marks)
 b) Produce table(s), drawings and arrows showing variable tracking and function call tracking-as rough work. (8 marks)
 Space available on last page for rough work (you can also use back of previous page if you wish)

```

1. #include <stdio.h>
2. #define CAP 5
3. int mix(char);
4. char chop(int[], int, int);
5.
6. //printPlate prints the plate's values
7. //separated by a space on one line
8. //when you see it called just
9. //print values in plate array
10. void printPlate(int plate[], int size);
11.
12. int main(void){
13.     int plate[CAP]={0};
14.     int apple = 6;
15.     int grape = 2;
16.     char pear;
17.     plate[0] = apple++;
18.     plate[1] = apple;
19.     plate[2] = (10 + 3 * 2) - 6 * 4;
20.     plate[3] = 15 % 4 + 14 / 3;
21.     plate[4] = grape += 2;
22.     printf("apple: %d\n", apple);
23.     printf("grape: %d\n", grape);
24.     printPlate(plate, 5);
25.
26.     pear = chop(plate, 2, 4);
27.     printf("pear: %c\n", pear);
28.     printPlate(plate, 5);
29.
30.     apple = mix(pear);
31.     printf("apple: %d\n", apple);
32.     return 0;
33. }
34. int mix(char fruit){
35.     int rc = 2;
36.     int i;
37.     while(rc % 4 != 0){
38.         for (i = 0; i < 4; i++){
39.             printf("%c-", fruit);
40.         }
41.         printf("\n");
42.         rc += 3;
43.     }
44.     return rc;
45. }

```

```

46. char chop(int plate[], int plum, int cherry){
47.     int tmp;
48.     char rc;
49.     if(plate[plum] > plate[cherry]){
50.         printf("first\n");
51.         tmp = plate[plum];
52.         plate[plum] = plate[cherry];
53.         plate[cherry] = tmp;
54.         rc = '!';
55.     }
56.     else{
57.         printf("second\n");
58.         plate[plum] += cherry;
59.         plate[cherry] += plum;
60.         rc = '*';
61.     }
62.     return rc;
63. }
64. void printPlate(int plate[], int size){
65.     int i;
66.     for (i = 0; i < size; i++){
67.         printf("%d ", plate[i]);
68.     }
69.     printf("\n");
70. }

```

Put your output here:

This page is extra space for your walkthrough.